

Supplementary Material for: Eliciting Self-Verification in Multimodal Reasoning Agents with Reinforcement Learning

Vishwas Sathish^{1*†}, Viresh Ranjan^{2*}, Xinliang Zhu², Arnab Dhua², and
Douglas Gray²

¹ University of Washington, Seattle
vsathish@cs.washington.edu

² Amazon, USA
{vireshr,xlzhu,adhua,douggray}@amazon.com

Contents

A	Overview	2
B	Datasets	2
C	Multimodal Search Tools	3
D	Experimental Setup	4
D.1	Base Model	4
D.2	Tool Interaction	4
D.3	Training Configuration	4
D.4	Evaluation Protocol	5
D.5	Test-Time Scaling	5
E	Tool-Use Analysis	7
E.1	Search Statistics	7
E.2	Verification and Filtering	8
F	Additional Results	8
G	Prompts	10
H	Limitations	10

* Equal contribution. † Corresponding author.

A Overview

This supplementary document provides additional details on the datasets, search tools, experimental setup and prompts used in the main paper. It also includes further analyses of judge-based evaluation and evidence verification, together with supplementary results on Qwen3-VL-8B and GPT-5.

B Datasets

Following Section 4 in the paper, we train on the FVQA training split released by MMSearch-R1 and evaluate on four additional benchmarks: InfoSeek, MMSearch, SimpleVQA, and LiveVQA [3, 5, 8, 9, 14]. These datasets are chosen to test complementary aspects of search-augmented multimodal question answering, including search calibration, evidence use, and transfer beyond the training set.

FVQA (training). We use FVQA as the primary RL training set [14]. It contains a useful mix of search-free and search-required examples, which is important for learning when search is needed rather than over-invoking tools. In our setup, FVQA also supports the search-aware reward in Section 3.2 of the paper: we perform an initial forward pass without tool use and use the resulting signal as a proxy label when shaping search decisions.

InfoSeek. We evaluate on InfoSeek as an additional in-distribution benchmark following the convention of MMSearch-R1 [3, 14]. FVQA is closely tied to the InfoSeek benchmark family, while InfoSeek itself remains broader in scope, making it a useful test of whether the learned behavior transfers beyond the specific FVQA training subset.

MMSearch. We evaluate on MMSearch as an out-of-distribution benchmark for multimodal web search [8]. Compared with FVQA and InfoSeek, MMSearch places greater emphasis on a mix of text only questions without images and multimodal questions, noisy evidence, and effective integration of search results.

SimpleVQA. We additionally evaluate on SimpleVQA as an out-of-distribution benchmark that broadens the evaluation beyond standard image-grounded factual QA [9]. It contains both image-grounded and text-centric questions, includes multiple-choice examples, and covers both English and Chinese, making it a useful test of transfer across question formats, modality balance, and language.

LiveVQA. We include LiveVQA as a recent out-of-distribution benchmark with stronger dependence on timely external knowledge [5]. This makes it a useful stress test of whether the learned search behavior remains effective when up-to-date retrieval matters more directly.

Benchmark scope. Taken together, these benchmarks provide a broad test bed for search-augmented multimodal question answering. They probe whether a model can combine image understanding with external retrieval, adapt its search behavior across different question types, and use retrieved evidence effectively to support the final answer. At the same time, the current benchmark suite remains centered on factual question answering with search and therefore does not fully cover the broader space of multimodal agent behavior, such as longer-horizon interaction, richer tool use, or persistent memory across steps. We view extending evaluation in these directions as an important avenue for future work.

C Multimodal Search Tools

Search tools. Our agent is equipped with two external tools: image search and text search. Image search is implemented with SerpAPI and returns visually relevant web results for the input image. Text search follows a search-parse-summarize pipeline. At each step, the model may answer directly, invoke image search, or generate a text query q for textual retrieval. When a text query is issued, SerpAPI returns a ranked list of webpages, Jina Reader converts the retrieved pages into LLM-readable markdown, and a Qwen3-32B model summarizes the content into short natural-language tool observations, typically around 300–500 words. The summarization model is served with vLLM on 8 x A100 GPUs.

Caching and efficiency. To reduce latency during RL training, we cache image search results for all FVQA training samples at the start of training. Text search is the main systems bottleneck in both wall-clock time and API cost. A typical SVRL run with 400 training steps, batch size 32, and 4 GRPO rollouts per example can trigger roughly 10,000 to 25,000 text-search calls, depending on the learned search frequency. To reduce this overhead, we cache SerpAPI retrieval results in DynamoDB using the generated text query as the primary key and the original question as a secondary key. When an exact query match is unavailable, we perform approximate matching and reuse cached results if the Jaccard similarity exceeds 0.85. In practice, this reduces text-search cost by up to 40%.

Returned evidence and noise. For each text search, we typically extract up to six webpage summaries and randomly sample between two and six summaries to return to the model. We found this preferable to always returning a fixed number of summaries, since a deterministic interface can encourage overfitting to a narrow evidence pattern. Retrieved summaries are also often noisy: some webpages are not robot-readable, some contain cookie or login interstitials, and some are only weakly related to the generated query. As a result, the returned tool outputs can be partially relevant or entirely unhelpful, which makes evidence filtering and verification central to the setting studied in Section 3 of the paper.

D Experimental Setup

In this section, we describe the base model, prompting format, training configuration, and evaluation protocol used in our experiments.

D.1 Base model

Our primary experiments use Qwen2.5-VL-7B-Instruct as the base multimodal backbone, which we finetune with SVRL and evaluate across all benchmarks [1, 14]. Following the setup in Section 4 of the paper, the model operates over short multi-turn trajectories with a maximum of three interaction steps. In each episode, the model may answer directly, invoke image search once, or invoke text search up to two times before termination. We use a maximum context budget of 8124 tokens. In practice, this limited horizon is partly a systems choice, since longer contexts reduce rollout throughput and the effective batch size on a single 8 x A100 node.

D.2 Tool Interaction

We use a ReAct-style prompting format to structure intermediate reasoning and tool use [14, 17]. Each tool call must be preceded by a reasoning block written as `<reason> ... </reason>`. The model may then emit one of the following actions:

1. `<search><img></search>` to invoke image search,
2. `<text_search> q1, ..., q5 </text_search>` to invoke text search
3. `<answer> ... </answer>` to terminate with a final answer.

After any tool call, the next turn must also output a binary verification mask of the form `<verify> 0,1,0,... </verify>` before the next action or final answer. During training, the model proposes five candidate text queries and one valid query is randomly sampled and executed. During standard inference, we use the same protocol. In the test-time scaling setting discussed later, all candidate queries may be executed and aggregated by majority vote. A representative trajectory without detailed contents is shown in Table S1. See also Figure 1 of the paper for a full example.

D.3 Training Configuration

All SVRL finetuning experiments are run on a single node with 8 x A100 GPUs. Unless otherwise noted, the main runs use batch size 32, GRPO group size 4, and 400 optimization steps. We keep KL control enabled and use exact-match correctness during training rather than an LLM judge, as discussed in Section 3 of the paper. The remaining optimization settings and reward coefficients are summarized in Table S2.

Table S1: Representative multi-turn trajectory format used by the model.

Round	Model output
Round 1	<reason> ... need to perform image search. </reason> <search></search>
Round 2	<reason> Sources 1 and 3 are helpful ... need to perform text search. </reason> <verify> 1,0,1,0 </verify> <text_search> q1, q2, q3, q4, q5 </text_search>
Final round	<reason> Sources 3 and 4 present ... </reason> <verify> 0,0,1,1 </verify> <answer> 18-19 days </answer>

The final trajectory reward follows Section 3 of the paper and combines exact-match accuracy, a format reward, and the multiplicative SVRL shaping terms:

$$r(\tau) = (1 - \alpha)r_{\text{acc}}(\tau)r_{\text{svrl}}(\tau) + \alpha r_{\text{fmt}}(\tau),$$

where

$$r_{\text{svrl}} = r_{\text{aware}} \cdot r_{\text{count}} \cdot r_{\text{qalign}} \cdot r_{\text{salign}}.$$

D.4 Evaluation protocol

During evaluation, final answers are scored with GPT-5.0 using a fixed system prompt, following the judge-based evaluation protocol adopted by MMSearch-R1 [14]. We also report exact-match (EM) accuracy as a stricter lexical metric. In our setting, GPT-5.0 judge-based evaluation is used as the primary reported metric, while EM serves as a complementary measure under stricter matching. All other evaluation settings follow the benchmark protocols described in the paper.

D.5 Test-time scaling details

We perform preliminary test-time scaling experiments on a 256-example subset of FVQA-test by increasing the text-search budget at inference time. For a given budget $K \in \{1, \dots, 15\}$, we allow up to K parallel text searches for each question. SVRL already proposes up to five candidate text queries within a single reasoning step, so increasing K first uses these candidate queries directly and then generates additional queries through repeated higher-temperature rollouts. Specifically, as the allowed number of searches increases, we sample additional search proposals with temperatures gradually increased from 0 to 1. Each query is then executed independently through the same retrieval pipeline, producing webpage summaries that are rolled out in parallel until a final answer is obtained. This results in between 1 and 15 candidate answers per question, depending on

Table S2: Main reward terms and hyperparameters used in SVRL finetuning. Reward notation follows Section 3 of the paper.

Item	Symbol	Value
Exact-match reward	r_{acc}	$\{0, 1\}$
Search-aware factor	r_{aware}	λ_{sa} or 1
Search-aware coefficient	λ_{sa}	0.1
Query-count factor	r_{count}	1 or $\max(\epsilon, \hat{k}/k)$
Query-count range		$[\epsilon, 1]$
Query-alignment factor	r_{qalign}	$[\epsilon, 1]$
Snippet-alignment factor	r_{salign}	$[\epsilon, 1]$
Format reward	r_{fmt}	$[0, 1]$
Format weight	α	0.1
Training steps		400
Global batch size	B	32
Dr.GRPO group size	G	4
Maximum context budget		8124
Maximum prompt length		4096
Maximum response length		2048
Actor learning rate	η	2e-6
Decay ratio		0.95
Warmup steps		45
Mini-batch size		32
Micro-batch size per GPU		8
KL coefficient	β	0.001
Entropy coefficient		0.0
Maximum interaction steps	T	3
Image search limit		1
Text search limit		2
Candidate text queries per call	k	5
Training reward		Exact match
Evaluation judge		GPT-5.0

the allowed search budget. Since each branch triggers additional retrieval and summarization calls, inference cost increases with the search budget.

For aggregation, in practice, we first check whether any answer appears multiple times under exact string matching, and if so, we return the most frequent answer. If no exact majority exists, we consider a word embedding for each candidate answer and use a nearest-neighbor style consensus rule over the candidate set. Concretely, for each answer embedding, we identify the closest other answer embeddings in the set and count how often each candidate lies near the rest. The final prediction is the answer that is closest to the largest number of other candidate answers. We used ChatGPT’s text-embedding-3-small in our experiments. Although simple, this procedure is effective for revealing test-time scaling behavior while remaining lightweight compared with more structured aggregation

Table S3: Search versus no-search behavior on FVQA. We report the fraction of examples on which the model invokes search, together with accuracy on the search and no-search subsets separately.

Metric	MMSearch-R1++	SVRL
Used search	1445 (80.3%)	1105 (61.4%)
Accuracy on search subset	51.0%	56.4%
No search	355 (19.7%)	695 (38.6%)
Accuracy on no-search subset	78.6%	79.4%
Overall accuracy	56.4%	65.3%

rules. We leave stronger aggregation methods and explicit filtering of incorrect retrieved snippets for future work.

E Tool-Use Analysis

In this section, we provide additional analysis of search and evidence-selection behavior, complementing the analysis results in Section 4 of the paper and Figure 5 of the paper.

E.1 Search Statistics

Figure 5 of the paper summarizes search calibration using search precision and search recall. Search recall measures how often the model invokes search on examples for which search is required, while search precision measures how often a search invocation occurs on an example that truly benefits from search. In this sense, recall captures whether the model searches when needed, whereas precision captures whether it avoids unnecessary search. The plots in Figure 5 aggregate results from four experiments in total, consisting of two FVQA runs and two InfoSeek runs. As discussed in Section 3 of the paper, the search-required labels are based on the search-aware proxy used during training, so these statistics should be interpreted as a behavioral analysis of the learned policy rather than as an absolute measure of search necessity.

We complement this analysis with query-diversity statistics at inference time. On FVQA, the model generates an average of 4.86 unique queries per text-search step. This shows that the policy does not collapse to a single repeated query template despite the fixed five-query interface, and supports the intended role of the query-count reward described in Section 3 of the paper.

Table S3 further separates FVQA performance into search and no-search subsets. Compared with MMSearch-R1++, SVRL invokes search much less often (61.4% versus 80.3%) while improving accuracy on both subsets, from 51.0% to 56.4% on the search subset and from 78.6% to 79.4% on the no-search subset. This indicates that the overall gain comes from better search calibration together

Table S4: Conditioned query analysis. Error rates after conditioning on query quality and retrieval success. Lower is better. Query quality and retrieval usefulness are annotated by GPT-5.0 with access to the ground-truth answer.

Condition	MMSearch-R1++	SVRL
Good query $\rightarrow$ wrong answer	38.4%	33.7%
Good query + useful result $\rightarrow$ wrong answer	33.8%	29.1%
Good query $\rightarrow$ no useful result	8.3%	6.9%

with stronger performance when search is used, rather than simply increasing tool use.

Finally, Table S4 reports a conditioned query analysis in which query quality and retrieval usefulness are annotated by GPT-5.0 with access to the ground-truth answer. Even after conditioning on cases where the issued query is judged to be good, SVRL still improves over MMSearch-R1++. In particular, it reduces answer error both when conditioning only on a good query and when further conditioning on retrieval of at least one useful result. It also slightly reduces the rate at which a good query fails to retrieve any useful result. Taken together, these results suggest that the gains from SVRL are not limited to better query generation, but also extend to stronger downstream evidence use.

E.2 Verification and Filtering

We finally analyze the verification masks emitted after each tool call by comparing the model’s binary `<verify>` outputs against GPT-5.0 filtering labels over the same retrieved summaries. Table S5 summarizes the results over all summaries jointly scored by both the model and GPT-5.0. Overall agreement is 71.3%, which indicates substantial alignment with the teacher signal used during training. However, the disagreements are strongly asymmetric. The model rarely over-filters, with only 82 cases where it rejects a summary that GPT-5.0 considers useful, but it under-filters much more often, with 1495 cases where it accepts a summary that GPT-5.0 labels as noise. This pattern suggests that the learned filter is conservative in discarding evidence and tends to retain questionable summaries rather than risk suppressing potentially useful information. We return to this asymmetry in the Limitations section below.

F Additional Results

In this section, we provide additional quantitative and qualitative results that complement those in the paper.

SVRL on Qwen3-VL-8B. We also finetune Qwen3-VL-8B using the same SVRL training procedure as in the main experiments. The results are reported in Table S6. Overall, SVRL-8B performs very similarly to SVRL-7B and substantially outperforms MMSearch-R1++ on both FVQA and InfoSeek under

Table S5: Alignment between SVRL’s verification mask and GPT-5.0 oracle filtering labels over retrieved summaries. The main disagreement is under-filtering: the SVRL finetuned model is much more likely to retain noisy evidence than to discard useful evidence.

Metric	Value
Total snippets scored by both	5490
Agreement b/w SVRL and GPT-5.0	3913/5490 (71.3%)
Both relevant (model = 1, GPT-5.0 = 1)	2851
Both noise (model = 0, GPT-5.0 = 0)	1062
Under-filtered (model = 1, GPT-5.0 = 0)	1495
Over-filtered (model = 0, GPT-5.0 = 1)	82

GPT-5.0 judge-based evaluation. In particular, SVRL-8B reaches 64.9 on FVQA and 64.0 on InfoSeek, compared with 56.4 and 55.1 for MMSearch-R1++. These results suggest that the gains from SVRL are not tied to a single backbone and transfer to a different multimodal base model under the same finetuning setup.

Comparison to GPT-5.2. Table S6 also reports inference-only results for GPT-5.2 and GPT-5.2-thinking as stronger proprietary reference models. A direct-answer GPT-5.2 baseline improves over direct-answer Qwen3-VL-8B, but remains far below search-based methods. Under adaptive search, GPT-5.2-thinking performs competitively, but both SVRL-7B and SVRL-8B achieve higher GPT-5.0 judge-based accuracy on FVQA and InfoSeek. This comparison is notable because it shows that the proposed training procedure can make compact open-source models competitive with, and in these settings stronger than, a much larger proprietary system when both are evaluated in a search-based regime.

Judge-based evaluation and exact match. The table also reinforces the distinction between judge-based evaluation and exact match. Across nearly all methods, exact-match accuracy is substantially lower than GPT-5.0 judge-based accuracy, especially for proprietary models. This gap is consistent with the discussion in the evaluation section: exact match is sensitive to surface-form variation, while the judge-based metric is more tolerant to paraphrases and semantically equivalent answers. Importantly, the overall ranking of the stronger search-based methods remains favorable to SVRL under both evaluation protocols.

Additional qualitative examples. Figure S1 presents additional qualitative examples covering both positive and negative scenarios. These examples provide a more detailed view of model behavior beyond final accuracy, including successful cases where search and verification lead to the correct answer, as well as failure cases where retrieval noise, imperfect filtering, or incorrect evidence use still lead to errors. Together, they illustrate both the strengths of the proposed approach and the main failure modes that remain.

Table S6: Accuracy (%) on FVQA and InfoSeek under GPT-5.0 judge-based evaluation and exact match (EM). We evaluate against the latest GPT-5.2 thinking and no-thinking modes. Additionally we report accuracies on SVRL finetuned Qwen-3-VL-8B which shows similar results to SVRL-7B presented in the main paper.

Model	FVQA		InfoSeek	
	GPT-5.0	EM	GPT-5.0	EM
Direct Answer				
Qwen3-VL-8B	22.2	15.4	22.2	13.9
GPT-5.2	34.0	14.3	29.9	12.4
Adaptive Search				
MMSearch-R1++	56.4	40.6	55.1	30.4
SVRL-7B	65.3	43.0	64.6	41.7
SVRL-8B	64.9	40.2	64.0	36.5
GPT-5.2-thinking	60.8	21.2	52.2	15.8

G Prompts

We follow the same overall prompt structure as MMSearch-R1 for training, inference, and judge-based evaluation [14], with minor changes to match the ReAct-style interaction format used in our method. Table S7 shows the main prompt used for search-based training and inference. For the direct-answer baseline and judge-based evaluation, we use the same prompt templates as in MMSearch-R1, with GPT-5.0 replacing the original judge model where applicable. Table S7 summarizes the system prompt used for search-based training and inference in SVRL.

H Limitations

Our method still has several limitations. First, although verification is performed by the agent itself at inference time, the query- and snippet-alignment rewards during training rely on verifier-generated labels with access to the ground-truth answer. This improves reward quality, but also introduces a dependence on the teacher model and its annotation biases. Similarly, our primary evaluation metric uses GPT-5.0 as a judge, which is more robust than exact matching for aliases and paraphrases but is not fully reproducible and can be sensitive to prompt phrasing. Future work could reduce this dependence by developing stronger self-supervised or process-level verification signals, lightweight learned verifiers, or label-free process rewards [2, 4, 10]. Second, the retrieval setting remains noisy and non-stationary: web pages can be irrelevant, inaccessible, outdated, or poorly parsed, and our appendix analysis shows that the learned filter is still more likely to retain noisy evidence than to discard useful evidence. Improving robustness under retrieval shift may require tighter integration of retrieval, reranking, and



Fig. S1: Qualitative examples with positive and negative scenarios.

verification, as well as training on richer web-agent settings with more realistic search dynamics [6, 7, 11, 14].

A second limitation is scope. Our experiments focus on search-augmented multimodal question answering with a short interaction horizon and a restricted action space centered on image search, text search, verification, and answer generation. As a result, the current results should be interpreted as evidence of improved search-based multimodal reasoning rather than a complete treatment of general multimodal agency. In addition, our test-time scaling experiments use a simple consensus-based aggregation rule over candidate answers. While this is sufficient to show that extra search budget can improve accuracy, more structured search and aggregation procedures may further strengthen performance [13, 15, 16]. Extending SVRL to richer tool suites, longer-horizon interaction, and stronger inference-time aggregation remains an important direction for future work [6, 7, 12].

References

1. Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., Ye, J., Zhang, X., Xie, T., Cheng, Z., Zhang, H., Yang, Z., Xu, H., Lin, J.: Qwen2.5-vl technical report. arXiv preprint arXiv:2502.13923 (2025), <https://arxiv.org/abs/2502.13923>
2. Chen, X., et al.: Free process rewards without process labels. arXiv preprint arXiv:2412.01981 (2024), <https://arxiv.org/abs/2412.01981>

Table S7: Main system prompt used for search-based training and inference in SVRL.

System message. You are a helpful assistant who will be evaluated on a Visual Question Answering Task. You should strictly follow a reason-to-act process to answer a user-provided question about an image. All thinking must be written inside <code><reason></code> and <code></reason></code> tags.
Available actions. First analyze the question and any available observations, including the user-provided image and any retrieved search results. Then choose one of the following actions: <code><search></search></code> to perform image search; <code><text_search>query1, query2, query3, query4, query5</text_search></code> to generate five unique relevant text queries, one of which is selected at random during training; or <code><answer>xxxx</answer></code> to return the final answer. If an action was performed previously, output a binary relevance list containing as many comma separated values as the number of tool output snippets like this: <code><verify>0/1, 0/1, ...</verify></code> before choosing the next action. Retrieved image and text results are placed inside <code><information></code> and <code></information></code> tags.
Allowed output formats. The model output must follow exactly one of the following formats: <pre> <reason> YOUR THINKING PROCESS </reason> <search></search> or <reason> YOUR THINKING PROCESS </reason> <verify>0/1, 0/1, ...</verify> <text_search> FIVE GENERATED COMMA SEPARATED UNIQUE TEXT QUERIES </text_search> or <reason> YOUR THINKING PROCESS TO ASSESS TOOL OUTPUTS AND ANSWER THE QUESTION </reason> <verify>0/1, 0/1, ...</verify> <answer> YOUR ANSWER AFTER GETTING ENOUGH INFORMATION </answer> </pre>
Additional instructions. If an image or text search call was made previously, the model should first reason about why each returned summary was helpful or unhelpful and then output a binary relevance list inside <code><verify></code> and <code></verify></code> before producing the final answer. If no prior search call was made, output an empty <code><verify> </verify></code> block. The number of binary scores must match the number of retrieved summaries. The final answer must appear only inside <code><answer></code> and <code></answer></code> tags, without extra explanation. If the question expects a year, date, quantity or location, be as specific as possible and mention appropriate units. If the question is yes-or-no, answer only yes or no.

3. Chen, Y., Hu, H., Luan, Y., Sun, H., Changpinyo, S., Ritter, A., Chang, M.W.: Can pre-trained vision and language models answer visual information-seeking questions? arXiv preprint arXiv:2302.11713 (2023), <https://arxiv.org/abs/2302.11713>
4. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021)
5. Fu, M., Peng, Y., Liu, B., Wan, Y., Chen, D.: LiveVQA: Live visual knowledge seeking. arXiv preprint arXiv:2504.05288 (2025)
6. Geng, X., Xia, P., Zhang, Z., Wang, X., Wang, Q., Ding, R., Wang, C., Wu, J., Zhao, Y., Li, K., Jiang, Y., Xie, P., Huang, F., Zhou, J.: Webwatcher: Breaking new frontier of vision-language deep research agent. arXiv preprint arXiv:2508.05748 (2025), <https://arxiv.org/abs/2508.05748>
7. Hu, Z., Iscen, A., Sun, C., Chang, K.W., Sun, Y., Ross, D.A., Schmid, C., Fathi, A.: Avis: Autonomous visual information seeking with large language model agent. arXiv preprint arXiv:2306.08129 (2023), <https://arxiv.org/abs/2306.08129>
8. Jiang, D., Zhang, R., Guo, Z., Wu, Y., Lei, J., Qiu, P., Lu, P., Chen, Z., Fu, C., Song, G., Gao, P., Liu, Y., Li, C., Li, H.: MMSearch: Benchmarking the potential of large models as multi-modal search engines. arXiv preprint arXiv:2409.12959 (2024)
9. Li, Y., et al.: SimpleVQA: A factuality-focused visual question answering benchmark. arXiv preprint arXiv:2501.01805 (2025)

10. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., Cobbe, K.: Let’s verify step by step. arXiv preprint arXiv:2305.20050 (2023)
11. Narayan, K., Xu, Y., Cao, T., Nerella, K., Patel, V.M., Shiee, N., Grasch, P., Jia, C., Yang, Y., Gan, Z.: Deepmmsearch-r1: Empowering multimodal llms in multimodal web search. arXiv preprint arXiv:2510.12801 (2025), <https://arxiv.org/abs/2510.12801>
12. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761 (2023), <https://arxiv.org/abs/2302.04761>
13. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2022)
14. Wu, J., Deng, Z., Li, W., Liu, Y., You, B., Li, B., Ma, Z., Liu, Z.: Mmsearch-r1: Incentivizing llms to search. arXiv preprint arXiv:2506.20670 (2025), <https://arxiv.org/abs/2506.20670>
15. Xie, Q., Narang, S., Jain, G., Balasubramanian, N., Le, Q.V.: Self-evaluation guided beam search for reasoning. In: Advances in Neural Information Processing Systems (NeurIPS) (2023), <https://openreview.net/forum?id=Bw82hwg5Q3>
16. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601 (2023)
17. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022), <https://arxiv.org/abs/2210.03629>